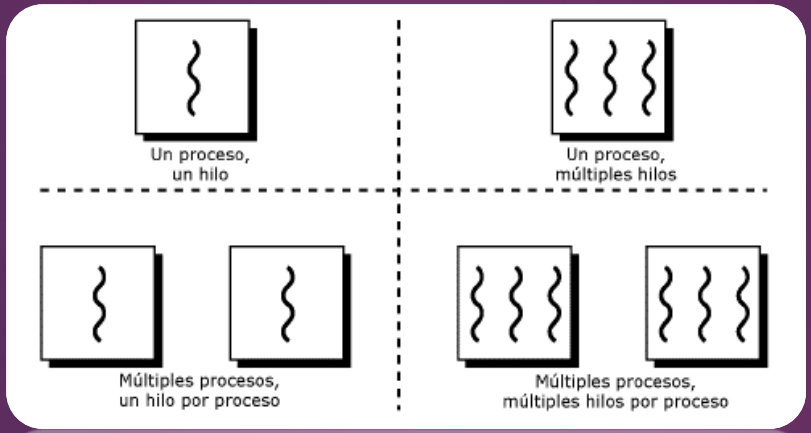
OS: HILOS



AUTOR: EZEQUIEL ESTEBAN

Introducción

- **Los sistemas modernos ejecutan múltiples tareas al mismo tiempo**
  - Navegar por internet
  - Reproducir música
  - Ejecutar aplicaciones
- **Problema:**

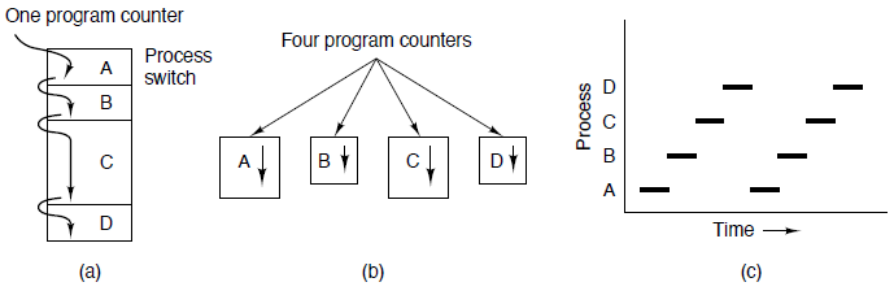
¿Cómo puede una CPU manejar varias tareas a la vez?

La solucion es la concurrencia que ofrece el sistema operativo

  - Administrar recursos
  - Planificar la ejecución de tareas

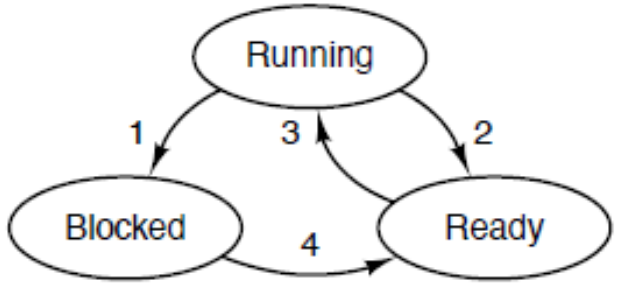
Repaso de Procesos

- Un **proceso** es un *programa en ejecución*.
- ***Incluye:***
  - Contador de programa (PC)
  - Registros
  - Memoria (código, datos, stack)



Cada proceso tiene su **propio espacio de direcciones**

- **Estados básicos:**
  - Running → ejecutándose
  - Ready → listo para ejecutar
  - Blocked → esperando (ej: entrada/salida)
- **El sistema operativo alterna entre procesos**



Limitaciones de los Procesos

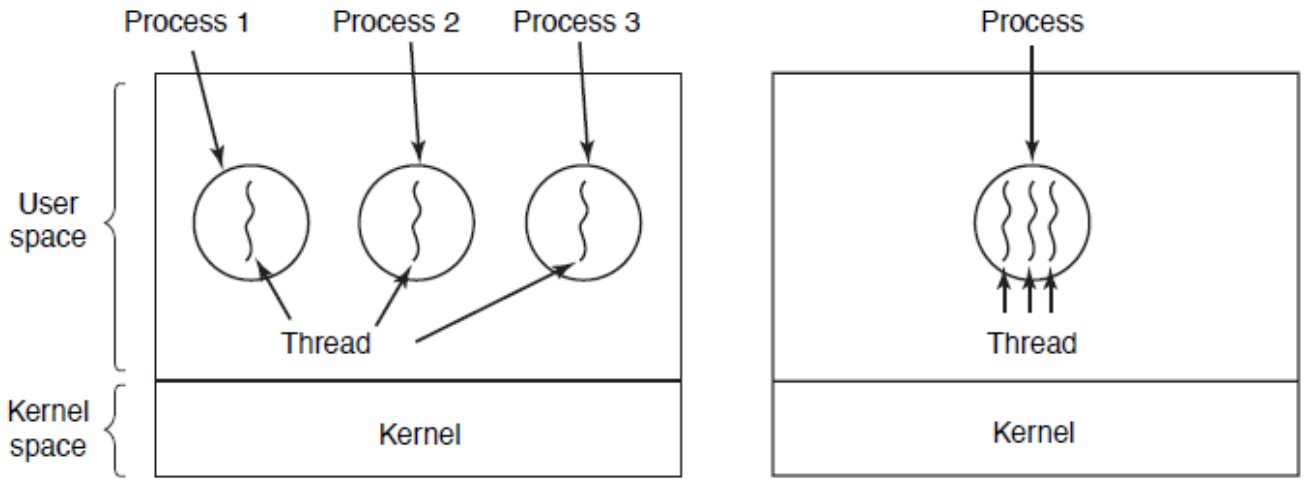
- **Son pesados:** Crear y eliminar procesos consume tiempo
- **Alto costo de cambio de contexto:** Guardar/restaurar estado completo
- **No comparten memoria fácilmente:** Cada proceso tiene su propio espacio
- **Comunicación compleja (IPC):** Pipes, colas de mensajes, etc.

- **Problema:**
- No son eficientes para tareas concurrentes relacionadas.
- Se necesita una alternativa mas liviana: ***Threads o Hilos***

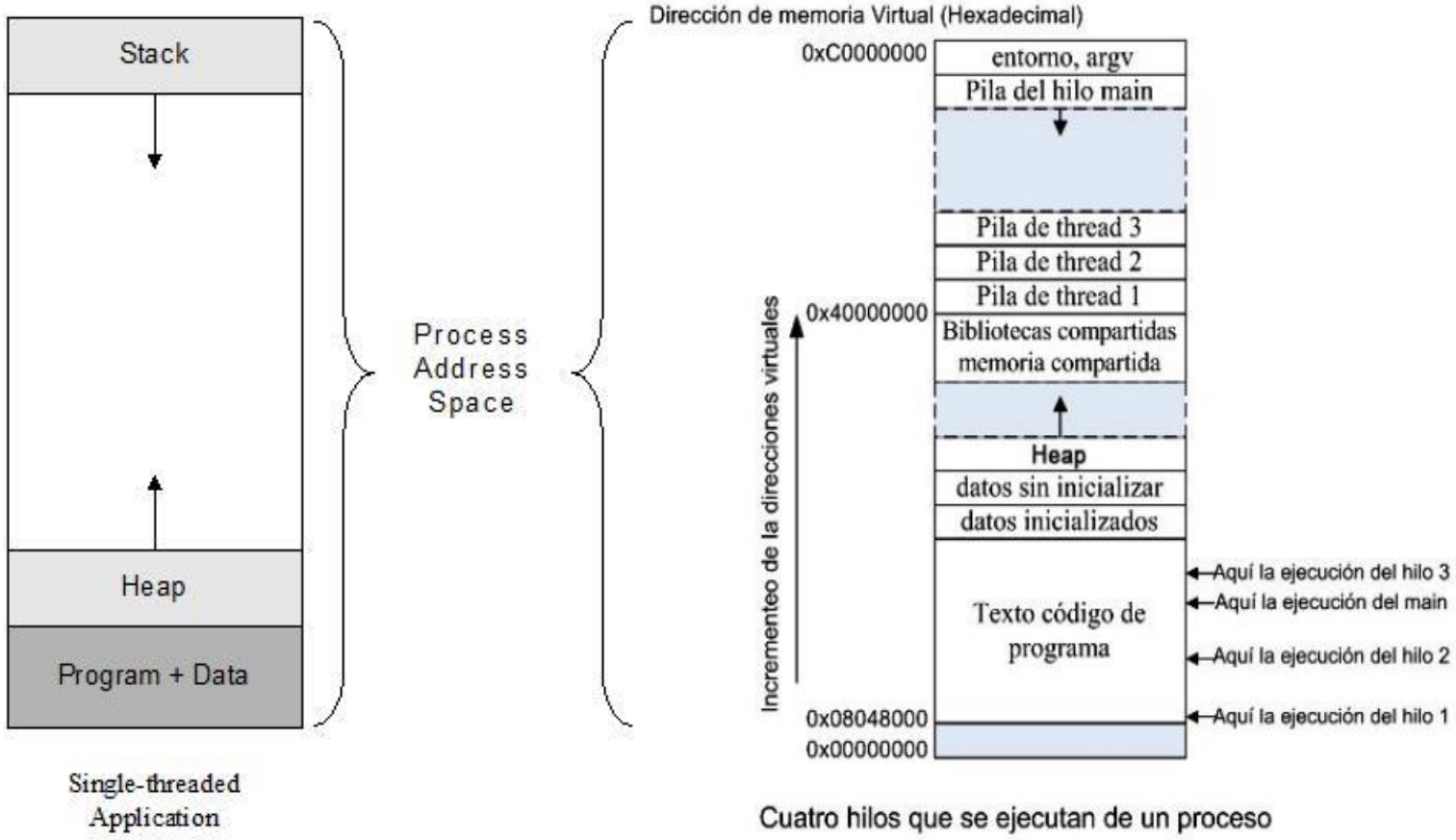
Hilos (Threads)

- **¿Qué es un Hilo?**

Un hilo (o thread) es la **unidad básica de ejecución** más pequeña que un sistema operativo puede gestionar. Actúa como un "proceso ligero**" dentro de un proceso mayor**, permitiendo que las aplicaciones realicen **múltiples tareas simultáneamente** (concurrencia), **compartiendo recursos como memoria y archivos**, lo que mejora la velocidad y eficiencia.



Hilos: Espacio de Memoria



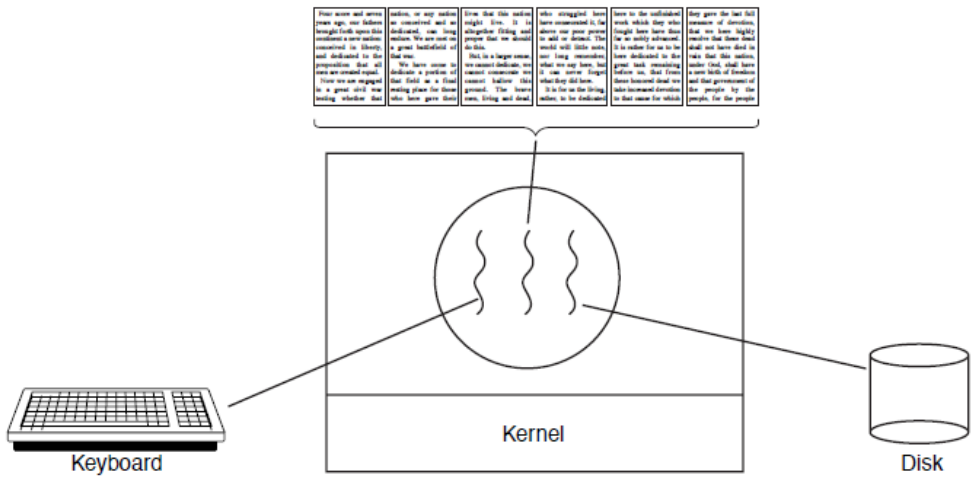
Cuatro hilos que se ejecutan de un proceso

Procesos vs Hilos

Procesos	Hilos
Unidad de ejecución independiente	Unidad de ejecución dentro de un proceso
Memoria propia	Memoria compartida
Comunicación compleja (IPC)	Comunicación simple
Creación costosa	Creación rápida
Cambio de contexto pesado	Cambio de contexto liviano

- **Hilos** son más eficientes para tareas relacionadas
- **Procesos** ofrecen mayor aislamiento

Ejemplo: Procesador de texto



Hilo principal: Maneja teclado y pantalla
Hilo de guardado automático: Guarda el documento cada cierto tiempo
Hilo corrector ortográfico: Revisa palabras en segundo plano

¿Por qué NO usar procesos en un procesador de texto?

1. **No comparten memoria fácilmente**
   - Cada proceso tiene su propia memoria
   - El documento estaría "duplicado" o habría que sincronizarlo
2. **Comunicación más costosa**
   - Procesos → usan **IPC**
   - Con procesos → lento y complicado
3. **Las tareas están fuertemente relacionadas**
   - Guarda, escribe, corrige
   - Tiene mas sentido que estén en el mismo espacio de memoria, por lo cual es mejor usar hilos

Ventajas y desventajas

Ventajas	Desventajas
Menor costo de creación y finalización	Problemas de sincronización
Cambio de contexto más rápido	Condiciones de carrera
Comparten memoria (comunicación simple)	Errores difíciles de detectar
Mejor uso de CPU (concurrencia)	Posibles bloqueos (deadlocks)
Mayor rendimiento en aplicaciones	Mayor complejidad de programación
Ideales para tareas relacionadas	Menor aislamiento entre tareas

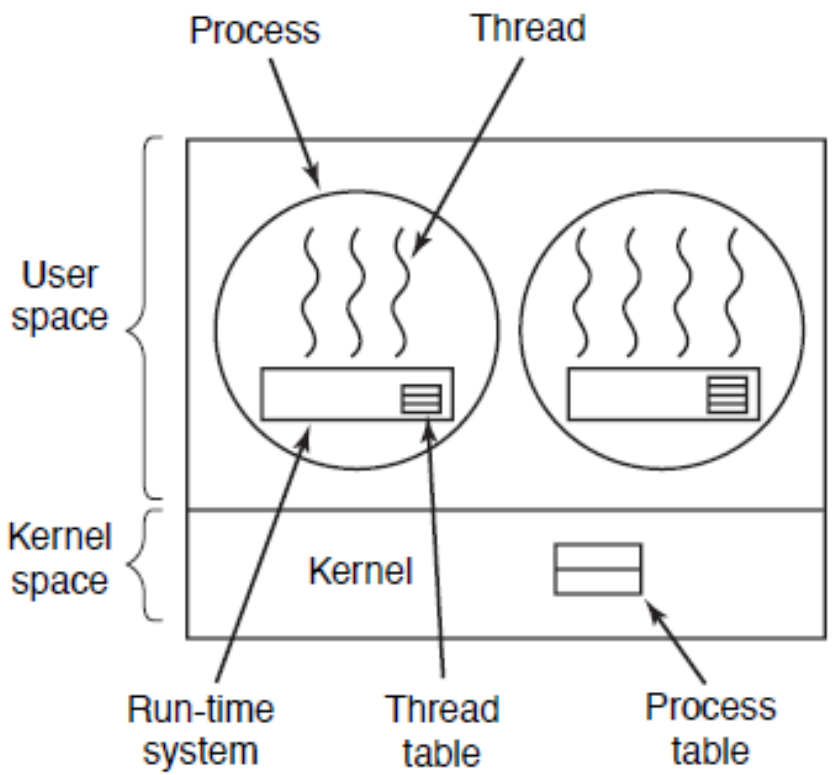
Implementación de Hilos

¿Quién gestiona los hilos: el usuario o el sistema operativo?

- Hilos en espacio de usuario (User-Level): gestionados por una libreria
- Hilos en el kernel (Kernel-Level): gestionado por el Kernel
- Modelo híbrido: combina ambos enfoques

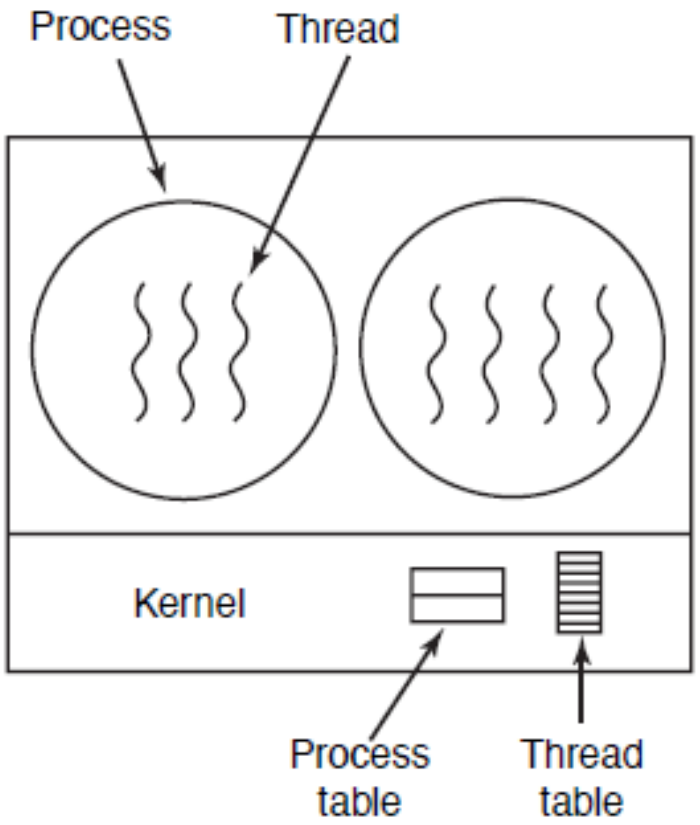
Hilos en espacio de usuario

- Los hilos se manejan **completamente en el espacio de usuario**, sin que el kernel se entere.
- **Una librería gestiona los hilos:** crea hilos, hace scheduling, guarda/restaura contexto
- No hay llamadas al sistema, por ende es mas rapidos.
- **Problema:** si un hilo hace una llamada bloqueante (ej read()), se bloquea todo el proceso ya que el kernel no sabe que existen otros hilos.
- Cada proceso tiene su **tabla de hilos** para mantener registro de los hilos que se ejecutan en ese proceso. Contiene el contador de programa, puntero de pila, registro, estado, etc. Esta tabla es manejada por el sistema de run time



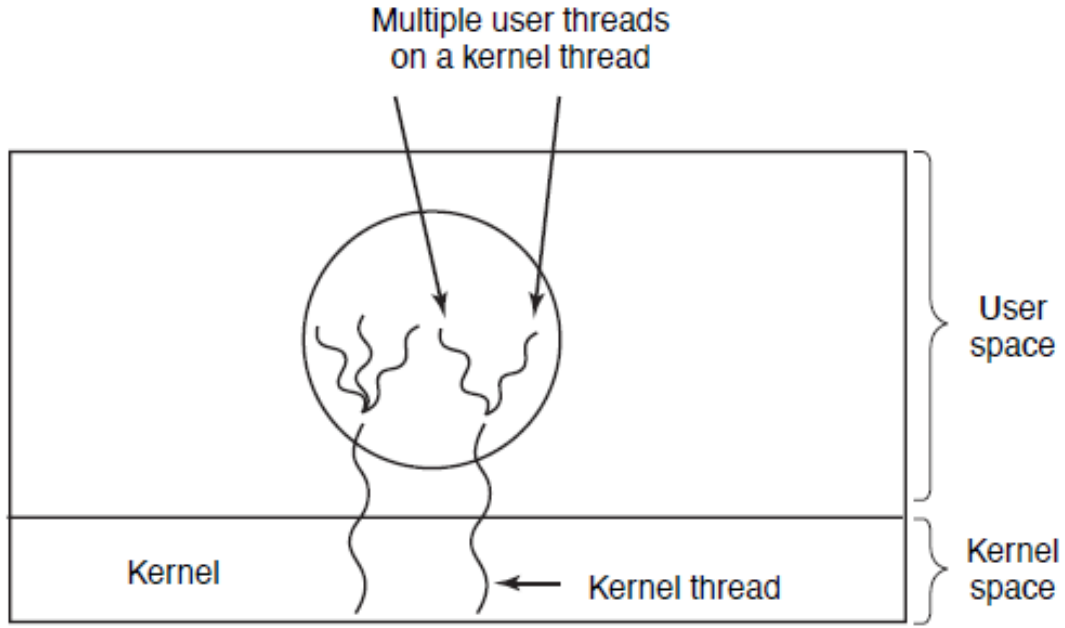
Hilos en espacio Kernel

- Los hilos son manejados directamente por el **sistema operativo.** El kernel sabe que hay múltiples hilos
- El scheduler del SO decide qué hilos ejecutar
- Se soluciona el problema de los bloqueos
- Desventaja: mas lento (hay syscalls), el overhead es mayor.
- Hay una sola tabla de hilos en el Kernel donde se almacena su creación o su destrucción



Modelo hibrido

- Combina lo mejor de ambos. Muchos hilos de usuario ↔ varios hilos de kernel.
- La librería crea muchos hilos y el kernel maneja un conjunto de hilos reales
- Ventajas: Flexible, buen rendimiento, evita bloqueos globales
- Desventajas: Muy complejo de implementar. Poco usado hoy en dia.



Hilos POSIX

- POSIX es un **estándar.** Define **cómo deben ser las funciones y comportamientos** de sistemas tipo UNIX.
- **POSIX Threads:** Es la **API estándar en C para manejar threads en sistemas POSIX.** Es una librería (pthread).
- Es una implementación en el esapcio Kernel.

Thread call	Description
Pthread_create	Create a new thread
Pthread_exit	Terminate the calling thread
Pthread_join	Wait for a specific thread to exit
Pthread_yield	Release the CPU to let another thread run

Gracias por su atencion